



Prof. Antoine Bosselut

Modern Natural Language Processing – CS-552

15.04.2026 from 11h30 to 13h00

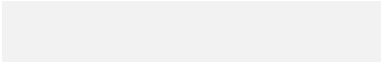
Duration : 90 minutes

1

Midterm 2026 Solutions

SCIPER: 111111

Room: INR 218

Signature: 

Do not turn the page before the start of the exam. This document is double-sided, has 24 pages, the last ones possibly blank. Do not unstaple.

- This is a closed book exam. Non-programmable calculators are allowed. No other electronic devices of any kind are allowed.
- Place on your desk: your student ID, writing utensils, one double-sided A4 page cheat sheet if you have one; place all other personal items below your desk.
- You each have a different exam.
- This exam has **multiple-choice** questions of varying difficulty. Each question is worth **one point**.
- Each question has **exactly one** correct answer. For each question, mark the box corresponding to the correct answer. You are not expected to get every question right even for the best grade.
- Only answers in this booklet count. No extra loose answer sheets. You can use the blank pages at the end as scrap paper.
- Use a **black or dark blue ballpen** and clearly erase with **correction fluid** if necessary.
- If a question turns out to be wrong or ambiguous, we may decide to nullify it.

Respectez les consignes suivantes Observe this guidelines Beachten Sie bitte die unten stehenden Richtlinien		
choisir une réponse select an answer Antwort auswählen	ne PAS choisir une réponse NOT select an answer NICHT Antwort auswählen	Corriger une réponse Correct an answer Antwort korrigieren
  		 
ce qu'il ne faut PAS faire what should NOT be done was man NICHT tun sollte		
     		



Classical LMs & RNNs

Question 1 In a vanilla RNN, the hidden state is computed as $h_t = \sigma(W_{hx} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$. When computing the gradient $\frac{\delta h_t}{\delta h_{t-T}}$ during backpropagation through time, the result is a product of T terms, each containing W_{hh} and the derivative of the activation function. Which of the following most accurately explains why using a sigmoid activation makes the vanishing gradient problem particularly severe?

- ☐ The sigmoid derivative is always exactly 0.5, which causes the gradient to halve at every time step.
- ☐ The sigmoid function is not differentiable at certain points, causing gradient computation to fail for long sequences.
- ☐ The sigmoid function saturates at 0 and 1, meaning its output is always less than 1, which directly causes the gradient to shrink.
- ☒ The sigmoid derivative has a maximum value of 0.25, so each multiplicative factor in the chain is at most 0.25, causing exponential decay over T steps.

Solution: The sigmoid derivative $\sigma'(u) = \sigma(u)(1 - \sigma(u))$ reaches its maximum of 0.25 at $u = 0$. Since $\frac{\delta h_t}{\delta h_{t-1}} = W_{hh} \cdot \sigma'(u)$, and $\sigma'(u) \leq 0.25$, the gradient is multiplied by a factor well below 1 at each step. Over T steps, this causes exponential decay.

Question 2 Karpathy et al. (2015) showed that individual dimensions of the LSTM cell state c can learn to track semantic features such as code indentation depth and whether text is inside quotation marks. Which of the following statements is **NOT FALSE** regarding how LSTM track long-range features?

- ☐ The LSTM uses a softmax over cell dimensions, forcing exactly one dimension to be active at a time, which creates a natural specialization.
- ☒ The element-wise (Hadamard) multiplication in $c_t = i_t \odot \tilde{c}_t + f_t \odot c_{t-1}$ allows each dimension of the cell state to be independently gated.
- ☐ The LSTM architecture includes an explicit feature assignment mechanism that maps semantic concepts to specific cell dimensions during initialization.
- ☐ The cell state is constrained to be binary (0 or 1), so each dimension naturally represents a boolean feature.

Solution: The $\odot$ operator in the LSTM equations is element-wise multiplication. This means each dimension of the cell state vector has its own forget gate value and input gate value. Dimension 42 might have $f_t[42] \approx 1$ (preserve) while dimension 100 has $f_t[100] = 0$ (reset). This per-dimension control allows different dimensions to operate at different timescales and learn to track different features, one for indentation, another for quotation state, etc.



Sequence-to-Sequence & Transformer Models

Question 3 Given the following pre-trained model configuration: `hidden_dim=1024`, `num_heads=8`, `intermediate_size=2048`, `num_layers=8`, `vocab_size=32,000` for a model with tied embeddings (the embedding and de-embedding matrices are shared), consider the following statements:

- A. In multi-head attention, each head projects the input into query, key, and value vectors of dimension $d_k = 1024/8 = 128$.
- B. The feed-forward network in each layer has **fewer** parameters than the multi-head attention sublayer, which includes all $W_{Q_i}, W_{K_i}, W_{V_i}$ projections across the 8 heads plus the output projection W_O .
- C. Self-attention has $\mathcal{O}(n^2 \cdot d)$ time complexity where n is sequence length and d is `hidden_dim`, making it the bottleneck for long sequences.
- D. Doubling `num_heads` to 16 while keeping `hidden_dim=1024` would double the total number of parameters in the attention layers.
- E. The embedding layer accounts for more than 30% of total parameters.

Which statements are correct?

- ☐ A, C, D, E
- ☐ A, B, C, E
- ☐ None of the options is correct.
- ☐ A, B, C
- ☒ A, C, E
- ☐ A, C, D

Solution: $1024/8 = 128$ (**A correct**). The n^2 factor in self-attention dominates for long sequences (**C correct**). The embedding matrix of $32,000 \times 1024 \approx 32\text{M}$ params, which accounts for more than 30% of the total model size of $\sim 96\text{M}$ (**E correct**). FFN has $1024 \times 2048 + 2048 \times 1024 \approx 4.2\text{M}$ params; attention has $4 \times 1024^2 \approx 4.2\text{M}$ params—they are *equal*, not fewer (**B wrong**). The W_Q, W_K, W_V, W_O matrices are each $[\text{hidden_dim} \times \text{hidden_dim}]$ regardless of `num_heads`; parameter count stays the same (**D wrong**).



Question 4 Based on your understanding of the Transformer architecture, consider the following statements:

- A. Replacing learned positional embeddings with the fixed sinusoidal encodings from the original Transformer paper would reduce the number of trainable parameters without affecting the model's ability to handle sequences up to a specified `max_len`.
- B. BERT uses only the encoder stack with bidirectional self-attention (no causal mask) and no decoder, making it **unsuitable** for autoregressive generation without modifications.
- C. In an encoder-decoder Transformer, removing the cross-attention sub-layer from decoder layers (keeping only the masked self-attention and feed-forward sub-layers) yields a decoder block functionally equivalent to a GPT-style decoder-only model.
- D. Pre-norm Transformers (applying LayerNorm **before** each sub-layer instead of after) are mathematically equivalent to the post-norm variant.

Which statements are correct?

- ☐ B, C
- ☐ None of the options is correct.
- ☒ A, B, C
- ☐ A, C, D
- ☐ A, B, D
- ☐ A, B, C, D

Solution: Sinusoidal encodings do not use parameters while learned position embeddings do (**A correct**). BERT is encoder-only with bidirectional attention, incompatible with autoregressive generation as-is (**B correct**). Stripping cross-attention from an encoder-decoder's decoder block leaves masked self-attention + FFN, i.e. a GPT-style block (**C correct**). Pre-norm and post-norm differ in gradient flow and training dynamics, so they are *not* mathematically equivalent (**D wrong**).

Question 5 In the Transformer decoder, masked multi-headed attention sets the attention scores for future tokens to $-\infty$ before applying softmax. A student proposes an alternative: instead of masking before softmax, simply set the attention scores to zero for future tokens after softmax. Which of the following is a reason why the original masking approach is preferable?

- ☐ Post-softmax zeroing preserves gradient flow to masked positions, which would allow information leakage during backpropagation.
- ☐ The original approach is only preferred for numerical stability; both methods produce identical forward-pass outputs.
- ☒ Setting scores to $-\infty$ before softmax is computationally cheaper because it avoids the renormalization step.
- ☐ Setting scores to $-\infty$ before softmax ensures the resulting attention weights over non-masked positions form a valid probability distribution after doing a separate renormalization step.

Solution: When we set $\alpha_{st} = -\infty$ before softmax, $e^{-\infty} = 0$, so these positions naturally receive zero probability and the softmax over remaining positions automatically sums to 1. Post-softmax zeroing would require explicit renormalization to maintain a valid distribution.

**Question 6**

In a standard Transformer encoder block, the feedforward network (FFN) applies the same two-layer MLP independently to each token position. A student argues that this sublayer is redundant because multi-headed self-attention already computes a unique representation for each token by attending over the full sequence. Consider the following statements about the role of the FFN:

- A. The FFN introduces nonlinearity into each token's representation independently, since self-attention cannot by itself model nonlinear feature interactions within a single position.
- B. The FFN allows tokens to exchange information across positions a second time within the same block, complementing the attention sublayer's cross-token mixing.
- C. Without the FFN, stacking multiple Transformer blocks would still only yield increasingly refined linear re-combinations of value projections.
- D. The FFN serves the same role as layer normalization: rescaling the residual connection's output to match the expected input distribution of the next block.

Which statements are correct?

- ☐ A, B
- ☐ A, B, C
- ☐ B, C, D
- ☒ A, C
- ☐ A, C, D

Solution: Self-attention computes a weighted sum of linearly projected value vectors. It does involve softmax as a non-linearity, but it does not operate token-wise. The FFN's ReLU is the only source of pointwise nonlinearity in the block (**A correct**). Without it, stacking blocks yields only linear re-combinations of value projections (**C correct**). The FFN operates per-position with no cross-token mixing (**B wrong**), and serves a fundamentally different role from LayerNorm's statistical rescaling (**D wrong**).



Tokenization

Question 7 Consider the following BPE training corpus with word frequencies:

"hug" (10), "pug" (5), "pin" (12), "buns" (4), "bugs" (5)

After initializing the character vocabulary with an end-of-word marker $\langle /w \rangle$, and running BPE for exactly **two** merge iterations, which pair could be merged in the **second** iteration?

- ☐ (u, $g\langle /w \rangle$)
- ☐ (u, g)
- ☐ (h, u)
- ☒ (ug, $\langle /w \rangle$)
- ☐ (p, i)
- ☐ (u, n)

Solution: In iteration 1, the pair (u, g) has the highest frequency (20) and is merged first. After this merge, recount pairs: (ug, $\langle /w \rangle$) has highest frequency of $10 + 5 = 15$, so it is merged second.

Question 8 Which of the following are valid reasons why subword tokenization (e.g., BPE) became the dominant approach for modern large language models?

- A. It guarantees that every token in the vocabulary corresponds to a linguistically meaningful morpheme.
- B. It provides a controllable trade-off between vocabulary size and sequence length.
- C. It mitigates the out-of-vocabulary problem by decomposing rare words into known subword units.
- D. It eliminates the need for positional embeddings because subwords encode positional information.
- E. It allows the same vocabulary to represent multiple languages when trained on multilingual corpora.

- ☐ C, D, E
- ☒ B, C, E
- ☐ A, B
- ☐ B, C
- ☐ A, D

Solution: A is false: BPE subwords do not necessarily correspond to morphemes. D is false: tokenization does not eliminate the need for positional embeddings. B, C, and E are valid advantages of subword tokenization.



Pretrained Language Models

BERT MLM Code:

```
1  import torch
2  import torch.nn.functional as F
3
4  def mask_tokens(input_ids, vocab_size, mask_token_id):
5      labels = input_ids.clone()
6      prob_matrix = torch.full(input_ids.shape, 0.15)
7      masked_indices = torch.bernoulli(prob_matrix).bool()
8
9      # Block A: Tokens NOT masked should have label -100
10     labels[~masked_indices] = -100
11
12     # Block B: 80% of masked tokens -> [MASK]
13     replace_mask = torch.bernoulli(
14         torch.full(input_ids.shape, 0.8)).bool() & masked_indices
15     input_ids[replace_mask] = mask_token_id
16
17     # Block C: 10% of masked tokens -> random token
18     replace_random = torch.bernoulli(
19         torch.full(input_ids.shape, 0.1)
20     ).bool() & masked_indices & ~replace_mask
21     random_tokens = torch.randint(vocab_size, input_ids.shape)
22     input_ids[replace_random] = random_tokens[replace_random]
23
24     # Block D: 10% of masked tokens -> keep original
25     # (no operation needed: these tokens were never modified above)
26
27     return input_ids, labels
```



Question 9 A colleague claims they have built a “truly bidirectional” language model by training a single LSTM that processes the input left-to-right and then right-to-left in two passes, concatenating the final hidden states. They argue this is equivalent to BERT’s bidirectionality. Consider the following statements:

- A. In this architecture, the representation of a token at position t in the forward pass has no access to information from positions $> t$, and vice versa for the backward pass. The concatenation happens too late for cross-directional interaction.
- B. BERT’s Transformer encoder allows each token to attend to all other tokens simultaneously within each layer, enabling genuinely bidirectional contextual representations.
- C. The colleague’s approach is actually superior to BERT because LSTMs have a recurrence inductive bias that Transformers lack.
- D. This is precisely the limitation identified in ELMo.
- E. If the colleague added cross-attention between the forward and backward passes at each LSTM layer, the architecture would become truly bidirectional.

Which of these statements are **TRUE**?

- ☐ B, D
- ☐ A, B, D, E
- ☒ A, B, D
- ☐ A, B, C, D
- ☐ B, C, D, E
- ☐ A, D

Solution: Each directional LSTM pass only sees one side of the context; concatenation cannot recover the cross-directional interactions. (**A correct**). Each layer of BERT’s self-attention has access to future and past context. (**B correct**). This is exactly ELMo’s known limitation (**D correct**). LSTM’s recurrence inductive bias does not make the architecture superior to BERT’s genuine bidirectionality, it is orthogonal to the claim of equivalence (**C wrong**). Adding cross-attention between passes is a plausible architectural improvement, but it would no longer match the colleague’s original claim that it is similar to BERT’s single-pass bidirectional attention (**E wrong**).



Question 10 Consider a sequence of 5 tokens $[t_1, t_2, t_3, t_4, t_5]$. Below are three attention mask matrices, where 1 means token i (row) can attend to token j (column) and 0 means it cannot.

$$M_A = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad M_B = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad M_C = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Consider the following statements:

- A. M_A is used in GPT-style autoregressive decoders; M_B is used in BERT-style bidirectional encoders; M_C could correspond to a prefix-LM where $[t_1, t_2, t_3]$ form the bidirectional prefix.
- B. M_A is used in BERT; M_B is used in GPT.
- C. M_C could represent the attention pattern in a model that encodes a prompt bidirectionally and then generates a continuation autoregressively, all within a single transformer.
- D. In BART, the *encoder* self-attention uses M_B and the *decoder* self-attention uses M_A . Cross-attention is a separate mechanism not captured by any of these three matrices.

Which statements are correct?

- ☐ A, D
- ☐ C, D
- ☐ B, C, D
- ☐ A, B, D
- ☐ A, C
- ☒ A, C, D

Solution: B is wrong (swaps GPT and BERT). The other three are correct characterizations.

Question 11 Consider the PyTorch-style pseudocode for BERT's Masked Language Modeling (MLM) above. Identify the line(s) containing a bug that would cause incorrect training behavior.

- ☐ Block C is incorrect: the random replacement probability of 0.1 is sampled from all tokens, meaning some already [MASK]-replaced tokens could also be randomly replaced, violating the mutual exclusivity of the 80/10/10 split.
- ☐ Block D is incorrect: it should explicitly copy the original tokens back for the remaining 10%.
- ☐ Block A is incorrect: labels for masked tokens should be set to -100 , not labels for unmasked tokens.
- ☒ There is no bug; the code correctly implements the 80/10/10 masking strategy.
- ☐ Block B is incorrect: the 0.8 probability should be applied only among the already-masked tokens, but here it samples from all tokens independently before intersecting, which changes the effective probability.

Solution: The code is correct. Independent Bernoulli sampling *before* intersection with `masked_indices` and `~replace_mask` preserves the 80/10/10 conditional split. The `&` operations enforce mutual exclusivity.



Training Data for LLMs

Question 12

You have built a pretraining dataset and want to evaluate its quality by training small ablation models. You train 1.4B-parameter models on 30B tokens from different dataset variants and compare them on a suite of benchmarks. Consider the following statements about selecting evaluation benchmarks for this purpose:

- A. Benchmarks performance should be *monotonically increasing* during training; a benchmark whose score oscillates wildly provides low signal for comparing dataset variants.
- B. For small models ($\sim 1-2B$), using “normalized log-likelihood” as the scoring metric tends to produce more reliable comparisons than generating the “letter answer” for multiple-choice questions.
- C. If models trained on two distributionally different datasets (e.g., The Pile and C4) achieve indistinguishable scores on a benchmark, that benchmark cannot reliably discriminate between dataset variants and should not be used for quality comparisons.
- D. You should use as many benchmarks as possible (e.g., 50+) to maximize statistical confidence, regardless of their individual signal quality.

Which statements are correct?

- ☒ A, B, C
- ☐ A, B, C, D
- ☐ A, B, D
- ☐ B, C, D
- ☐ A, C, D

Solution: A, B, C are correct. High-signal benchmarks should be monotonically increasing during training (A). Small models ($\sim 1-2B$) produce more reliable comparisons with normalized log-likelihood than letter-answer accuracy (B). A benchmark that cannot distinguish distributionally different datasets like The Pile vs. C4 lacks discriminative power (C). **D is incorrect.** Maximizing the number of benchmarks regardless of signal quality introduces noise rather than confidence. The datasets lecture from Hugging Face (Week 4) emphasizes selecting a few high-signal benchmarks over many noisy ones.



Question 13 You are debugging a perplexity-based quality filter. The filter uses a 5-gram language model trained on English Wikipedia. Documents with perplexity above a threshold τ are removed. After applying this filter to your web crawl, you observe that:

- Almost all non-English documents are removed (expected).
- Scientific papers with heavy mathematical notation are removed.
- Code-switching documents (e.g., English/Spanish mixed) are removed.
- Conversational forum posts are removed at a higher rate than news articles.

Which of the following **probably explain** these observations?

- A. The Wikipedia-trained LM assigns high perplexity to mathematical notation because LaTeX commands and symbols are rare in Wikipedia's running text.
- B. Code-switching documents are removed because the model's vocabulary and n-gram statistics reflect monolingual English Wikipedia, making mixed-language n-grams extremely unlikely.
- C. Forum posts are removed more often because their informal register, slang, and non-standard punctuation yield higher perplexity under a Wikipedia-trained model.
- D. Lowering τ would fix all of these issues by allowing more documents through the filter.
- E. This filter introduces a systematic bias toward encyclopedic-style text, which may reduce the diversity of the final training corpus.

- ☐ B, C, D, E
- ☐ A, C, E
- ☐ A, B, C, D, E
- ☒ A, B, C, E
- ☐ A, B, D

Solution: **A, B, C are correct** because the Wikipedia-trained LM assigns high perplexity to any content whose vocabulary or register diverges from encyclopedic English: mathematical notation produces rare n-grams (A), code-switching introduces out-of-vocabulary foreign tokens (B), and informal forum language contains n-grams absent from Wikipedia (C). **E is correct** because A–C collectively reveal a systematic bias toward Wikipedia-like text, reducing corpus diversity. **D is incorrect** because lowering τ makes the filter *stricter* (fewer documents fall below a lower threshold), removing *more* content.



LLM Prompting and Post-Training

Question 14 In the AutoPrompt algorithm, the goal is to automatically discover trigger tokens that increase the probability of predicting the correct label using a BERT-style masked language model (MLM). The prompt is constructed using a template: $x_{\text{prompt}} = \lambda(x_{\text{inp}}, x_{\text{trig}}, [\text{MASK}])$, where the task input sentence x_{inp} (e.g., "a real joy.") and trigger tokens x_{trig} are inserted into a template λ containing a [MASK] token. The MLM predicts a label token y at the mask position. During optimization, AutoPrompt constructs a candidate set of replacement tokens V_{cand} for each trigger token by searching over the MLM vocabulary V using a gradient-based approximation. Each token $w \in V$ has an input embedding w_{in} . Which of the following equations correctly defines the candidate set of trigger tokens used in AutoPrompt?

- ☒ $V_{\text{cand}} = \text{top-k}_{w \in V} (w_{\text{in}}^{\top} \nabla \log p(y | x_{\text{prompt}}))$
- ☐ $V_{\text{cand}} = \text{top-k}_{w \in V} (w_{\text{in}}^{\top} \nabla p(y | x_{\text{prompt}}))$
- ☐ $V_{\text{cand}} = \text{top-k}_{w \in V} (w_{\text{in}}^{\top} \nabla \log p(y | x_{\text{inp}}))$
- ☐ $V_{\text{cand}} = \text{top-k}_{w \in V} (\nabla \log p(y | x_{\text{inp}}))$

Solution: $\nabla \log p(y | x_{\text{prompt}})$ points in the direction in embedding space that would most increase the likelihood of predicting the correct label. $w_{\text{in}}^{\top} \nabla \log p(y | x_{\text{prompt}})$ answers: "how well does this candidate trigger token w align with the direction that improves the prediction?" A higher dot product means that trigger token is a better replacement.

Question 15 In RLHF, reward models are commonly trained using pairwise preference comparisons derived from human feedback. Consider a reward model $r_{\theta}(y | x)$ that assigns a scalar score to a completion y given a prompt x . Using the Bradley–Terry preference model, the probability that a human prefers completion y_c (chosen) over completion y_r (rejected) is modeled as

$$P(y_c > y_r | x) = \frac{\exp(r_{\theta}(y_c | x))}{\exp(r_{\theta}(y_c | x)) + \exp(r_{\theta}(y_r | x))}$$

Maximizing the log-likelihood of human preferences yields the reward model loss

$$\mathcal{L}(\theta) = -\log \sigma(r_{\theta}(y_c | x) - r_{\theta}(y_r | x))$$

The following statements are labeled A–D. Which option correctly identifies the set of true statements?

- A. The loss depends only on the difference $r_{\theta}(y_c | x) - r_{\theta}(y_r | x)$, implying that adding a constant offset to all reward scores leaves the preference probabilities unchanged.
- B. The loss guarantees that the learned reward model estimates the absolute human utility of a response, allowing direct comparison of rewards across different prompts.
- C. The Bradley–Terry formulation requires that the reward scores be normalized probabilities satisfying $\sum_y r_{\theta}(y | x) = 1$.
- D. The loss enforces that the reward of the preferred completion must always be strictly greater than the rejected completion after training converges.

- ☐ All except C
- ☐ None of the options is correct
- ☐ C only
- ☐ A and B
- ☒ A only
- ☐ B and D
- ☐ A and D



Solution: **A is correct** because the loss clearly only depends on the difference of rewards between preferred and rejected completions. **B is incorrect** as the reward models learn relative preferences, not absolute utilities across prompts. **C is incorrect** as the reward scores are unbounded scalars, not normalized probabilities. **D is incorrect** because the model encourages higher scores for preferred responses but does not guarantee strict inequality, especially under noisy labels.

Question 16 In standard RLHF, PPO optimizes a learned scalar reward with online policy updates, while DPO directly optimizes pairwise preferences using

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{P_{\text{curr}}(Y^+ | x)}{P_{\text{base}}(Y^+ | x)} - \beta \log \frac{P_{\text{curr}}(Y^- | x)}{P_{\text{base}}(Y^- | x)} \right).$$

Which one of the following statements is NOT true?

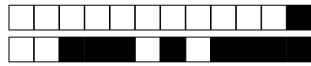
- ☐ A disadvantage of vanilla DPO is that, without rollout-based reward scoring, it tends to treat preference pairs more uniformly, even though some pairs may contain richer learning signal than others.
- ☐ Compared with PPO, vanilla DPO is less flexible because it directly optimizes pairwise preference comparisons rather than a more flexible scalar reward over newly sampled trajectories.
- ☐ Another disadvantage of DPO-like methods is that their training signal often comes from completions generated by previous or other models, which is one reason they may have a lower ceiling than online RLHF methods.
- ☒ Compared with PPO, a key disadvantage of DPO is that it can only learn from binary winner–loser comparisons and therefore cannot recover the same optimal policy as reward-based RLHF.

Solution: The theoretical motivation of DPO is precisely that, under certain assumptions, preference optimization can be rewritten into a direct policy objective without explicitly training a reward model first. So while DPO is often less flexible and may be weaker in practice than PPO-based RLHF, it is not correct to claim that binary preference data makes equivalence impossible. Other options are true statements.

Question 17 Which of the following statements about test-time scaling is NOT correct?

- ☒ Test-time scaling necessarily requires access to an external verifier or reward model; without such auxiliary components, extra inference-time compute cannot improve the model’s final answer.
- ☐ If a test-time scaling method consistently improves performance, this implies that the base model already internally contains useful partial competence that can be elicited or amplified by additional inference-time computation.
- ☐ Test-time scaling can improve accuracy even with fixed model parameters by increasing inference-time compute, for example through repeated sampling or self-verification.
- ☐ Unlike training-time scaling, test-time scaling allocates compute per input, enabling the model to spend more resources on difficult problems and less on straightforward ones.

Solution: External verifiers or reward models can help, but they are not necessary. Test-time scaling can still work through methods such as self-consistency, repeated sampling, search, longer reasoning traces, or iterative refinement without any separate verifier. Other options are true statements.



Neural Classifiers and Embeddings

Question 18 Pretrained embedding spaces often enable analogy arithmetic as seen in Week 1's exercise. The following statements are labeled A–E. Which option correctly identifies the set of true statements?

- A. `king - man + woman` $\approx$ `queen`.
- B. `france - paris` is the “capital” vector in embedding space.
- C. `doctor - man + woman` is necessarily unbiased.
- D. Analogy arithmetic reflects approximate non-linear relational structure in embedding spaces.
- E. Strong analogy performance implies the embeddings are unbiased.

- ☐ B only
- ☐ B and D
- ☐ A, B, and D
- ☐ A, C, and E
- ☐ None of the other options is correct.
- ☐ A only
- ☐ C, D, and E
- ☒ A and B

Solution: The true statements are **A and B**. Many pretrained embedding spaces support approximate analogy arithmetic, including country–capital style relations. However, these structures do not guarantee fairness: socially biased relations may also be encoded (**C and E incorrect**), and analogy behavior is typically discussed as an approximately linear geometric phenomenon rather than a non-linear one (**D incorrect**).

Question 19 Different tasks may require different model choices. Based on your understanding, which of the following statements regarding task:model pairings make sense?

- A. A sum-pool (or avg-pool) sentence representation plus a classifier can be used for sentiment prediction.
- B. An RNN without any pooling operation can serve for a review classification application.
- C. An RNN with token-level predictions can tag parts of speech in sentences.
- D. To enable text generation, we compose token representations, then apply softmax over the vocabulary.
- E. Logistic regression on sum-pooled input embeddings can be used as a general solution for text generation.

- ☐ None of the other options is correct.
- ☐ A, B, and D
- ☐ B, C, and E
- ☐ B and E
- ☐ A and C
- ☐ A, D, and E
- ☒ A, C, and D
- ☐ C and D

Solution: Sentence classification needs a sequence-to-label classifier (**A correct**), token tagging needs token-level outputs (**C correct**), and generation requires a next-token distribution over the vocabulary con-



ditioned on context (**D correct**). An RNN without an appropriate pooling cannot be used for classification (**B incorrect**), and a static sum-pooled logistic-regression setup is not a general formulation for text generation (**E incorrect**).

Question 20 When building a neural NLP system, you must choose a finite vocabulary even though natural language has a long tail of rare tokens (Zipf's law). Consider the implications of vocabulary truncation, <UNK> mapping, and tokenization strategy for representation learning and downstream generalization. The following statements are labeled A–E. Which option correctly identifies the set of true statements?

- A. Increasing vocabulary size always improves downstream generalization.
- B. Under Zipf's law, most token types are rare, so many embeddings are weakly estimated.
- C. Mapping rare or unseen tokens to <UNK> trades lexical specificity for statistical efficiency.
- D. If <UNK> is used, the model can no longer process unseen text.
- E. Better tokenization or subwording can reduce how much of the tail collapses into <UNK>.

- ☐ B, D, and E
- ☐ E only
- ☐ B and C
- ☐ A, B, and E
- ☒ B, C, and E
- ☐ A, C, and E
- ☐ C and D
- ☐ None of the other options is correct.
- ☐ B, C, and D

Solution: Natural language is long-tailed, so many token types are rare and their embeddings are weakly estimated (**B correct**). Mapping rare tokens to <UNK> sacrifices lexical specificity in exchange for better statistical efficiency (**C correct**). Subword tokenization reduces how often the long tail must collapse into <UNK> (**E correct**). Increasing vocabulary size does not *always* improve generalization (**A incorrect**), and using <UNK> does not prevent (rather facilitates) the model from processing unseen text (**D incorrect**).



Question 21 Noah, a student taking CS-552, trains embeddings using a supervised review-sentiment objective. Lucie trains embeddings with self-supervised learning. The following statements are labeled A–D. Which option correctly identifies the set of true statements?

- A. Noah must acquire a lot of labeled data.
- B. The embeddings Noah gets will reliably transfer to part-of-speech (POS) tagging.
- C. Lucie’s embeddings are guaranteed better than Noah’s for every downstream task.
- D. Lucie’s approach is more aligned with distributional semantics (“you know a word by the company it keeps”).

- ☐ A only
- ☐ B and C
- ☐ A, B, and D
- ☐ None of the other options is correct.
- ☒ A and D
- ☐ B, C, and D
- ☐ A and C
- ☐ D only

Solution: Supervised sentiment training is label-hungry (**A correct**). Self-supervised context prediction is more aligned with distributional semantics and tends to produce more broadly reusable lexical representations (**D correct**). There is no guarantee that Noah’s embeddings reliably transfer to POS tagging because supervised training on the sentiment classification task may over-specialize to that objective (**B incorrect**), nor that Lucie’s are uniformly best for every downstream task (**C incorrect**).

Question 22 Both CBOW and Skip-gram are self-supervised local-context objectives, but they differ in prediction direction and context aggregation. The following statements are labeled A–E. Which option correctly identifies the set of true statements?

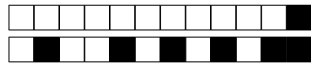
- A. Only CBOW uses explicit context aggregation (e.g., sum or average of surrounding words) to predict the target.
- B. Only Skip-gram involves multiplication with a learned embedding or output matrix.
- C. Only CBOW relies on surrounding context.
- D. The embeddings learned by CBOW and Skip-gram are as contextual as the final hidden states of BERT.
- E. Both learn from local context windows via self-supervised prediction objectives.

- ☐ A only
- ☒ A and E
- ☐ A, B, and E
- ☐ B and C
- ☐ B and D
- ☐ E only
- ☐ C, D, and E
- ☐ None of the other options is correct.

Solution: CBOW predicts a center word from an aggregated context, whereas Skip-gram predicts context



words from the center word. (**A correct**) Both are self-supervised local-context learning objectives (**E correct**) and both involve learned parameters; it is therefore false that only one uses learned matrices (**B incorrect**); or only one relies on context (**C incorrect**). The embeddings learned by CBOW and Skip-gram are independent of the final context they appear in unlike BERT (**D incorrect**).



Dataset Biases and Evaluation

Question 23 You aim to build a better benchmark by reducing biases. Which of the following strategies will you adopt for it?

- A. Once I have the data, I will train shortcut-seeking classifiers on different data splits, evaluate them on held-out examples, and remove examples that are consistently easy for these classifiers to predict.
- B. Once I have the data, I will train classifiers on different splits of the data, evaluate them on validation examples, and remove the examples that are solved by most classifiers, determined by majority voting.
- C. I will add contrast sets like ("The movie was surprisingly good.", "I did not like that movie.").
- D. For data annotation, I will prioritize the annotators that do not agree with others as they often bring unique perspectives.
- E. I will run pilot studies, and qualify the best crowd workers primarily based on their prior experience.
- F. I will add more data from a specific domain, since larger in-domain datasets automatically reduce spurious correlations.

- ☐ A, C, and D
- ☐ A and B
- ☐ B only
- ☐ F only
- ☐ C only
- ☐ All except B
- ☐ All except F
- ☐ C and F
- ☒ A only
- ☐ A and F
- ☐ C and E
- ☐ None of the other options is correct
- ☐ B and F

Solution: **Statement A** describes adversarial filtering, which removes examples that are too easily solved by a classifier and are therefore likely to contain exploitable artifacts. **B is incorrect** because the Week 5 lecture on dataset bias mitigation does not advocate majority-vote removal across classifiers. **C is incorrect** because the example pair is not a controlled contrast set. **D is incorrect** because persistent disagreement is not something to prioritize in annotator selection. **E is incorrect** because worker qualification should be based on pilot tasks or gold examples rather than primarily on prior experience. **F is incorrect** because simply adding more in-domain data does not automatically remove spurious correlations.



Text Generation and Evaluation

Question 24 Suppose you combine model and phrase-retrieval distributions as

$$p_{\text{mix}} = \lambda p_{\text{model}} + (1 - \lambda) p_{\text{phrase}}$$

and then rerank multiple decoded candidates. The following statements are labeled A–E. Which option correctly identifies the set of true statements?

- A. If $\lambda = 1$, phrase retrieval has no effect.
- B. If retrieval statistics are domain-mismatched, generation quality can degrade.
- C. Any convex mixture is guaranteed to outperform the base model.
- D. Reranking can target attributes beyond perplexity, such as style or factuality.
- E. Beam search makes reranking unnecessary.

☐ B, C, and D

☐ None of the other options is correct.

☐ A, D, and E

☐ C only

☐ B and E

☒ A, B, and D

☐ A and B

☐ A, B, C, and D

Solution: Setting $\lambda = 1$ ignores phrase retrieval entirely (**A correct**). Retrieval can help or hurt depending on how well its statistics match the target domain (**B correct**), and reranking can optimize attributes that are not captured by raw language-model likelihood, such as style or factuality (**D correct**). There is no guarantee that every convex mixture improves over the base model (**C incorrect**), and beam search does not make reranking unnecessary (**E incorrect**).



Question 25 Compute BLEU-2 with equal weights $w_1 = w_2 = 0.5$, using modified n -gram precision, and ignoring brevity penalty. You may use the following definition for BLEU-2:

- BLEU-2 combines modified unigram precision and modified bigram precision.
- “Modified precision” means an n -gram in the candidate only counts up to the number of times it appears in the reference.
- Unigrams and bigrams are weighted equally.
- The final BLEU-2 score is the weighted geometric mean of the unigram precision and the bigram precision.
- Note that the weighted geometric mean of a set $\{x_1, x_2, \dots, x_n\}$ with weights $\{w_1, w_2, \dots, w_n\}$ is given by:

$$\prod_{i=1}^n x_i^{w_i}$$

Reference: the cat is on the mat

Candidate: the cat is on mat

What is the BLEU-2 score (approximately)?

- ☒ 0.87
- ☐ 1.00
- ☐ 0.82
- ☐ 0.75
- ☐ 0.91

Solution: Candidate unigrams: the, cat, is, on, mat. All appear in the reference with valid clipped counts, so

$$p_1 = \frac{5}{5} = 1.$$

Candidate bigrams: the cat, cat is, is on, on mat. Matching reference bigrams are the cat, cat is, is on, but on mat does not match (reference has on the, the mat), so

$$p_2 = \frac{3}{4} = 0.75.$$

Without brevity penalty:

$$\text{BLEU-2} = \exp\left(\frac{1}{2} \log p_1 + \frac{1}{2} \log p_2\right) = \sqrt{p_1 p_2} = \sqrt{0.75} \approx 0.866.$$

So the closest option is **0.87**.



Question 26 You evaluate two decoding strategies for the same language model by computing perplexity on their own generated continuations. The following statements are labeled A–E. Which option correctly identifies the set of true statements?

- A. Self-perplexity can unfairly favor decoding strategies that explicitly chase high-probability tokens.
- B. Using self-perplexity is circular because the same model distribution is used both to generate and score outputs.
- C. Using self-perplexity is equivalent to human evaluation when references are unavailable.
- D. Low self-perplexity guarantees non-repetitive text.
- E. Perplexity on reference text is still useful for calibration assessment.

☒ A, B, and E

☐ A and B

☐ A, B, C, and E

☐ B, C, and E

☐ B and D

☐ None of the other options is correct.

☐ C only

☐ A, D, and E

Solution: Self-perplexity can favor decoders that seek high-probability continuations under the same model that later scores them (**A correct**), which makes the evaluation circular (**B correct**). This does not make it equivalent to human evaluation (**C incorrect**), and low self-perplexity does not guarantee diverse or non-repetitive outputs (**D incorrect**). Perplexity on held-out reference text remains useful for assessing calibration to real text (**E correct**).

Question 27 Consider the following statements labeled A–E in the context of using n -gram overlap metrics for open-ended generation tasks. Which option correctly identifies the set of true statements?

- A. They tend to degrade as tasks become more open-ended.
- B. They can give false negatives when semantically valid paraphrases have low lexical overlap.
- C. They can give false negatives for lexically similar but semantically wrong outputs.
- D. They are invariant to output length.
- E. They are often used for their reliable output.

☐ C, D, and E

☐ None of the other options is correct.

☐ B and D

☐ A, C, and E

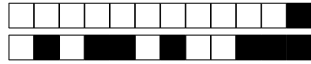
☐ A, B, and D

☒ A and B

☐ B only

☐ A only

Solution: As generation tasks become more open-ended, simple lexical overlap correlates less reliably with actual meaning quality (**A correct**). This can produce false negatives when a good paraphrase uses different wording (**B correct**). Lexically similar but semantically wrong outputs are better described as false



positives (**C incorrect**), these metrics are not invariant to length (**D incorrect**), and they are often used because they are cheap and fast rather than because they are uniformly reliable (**E incorrect**).



Text Generation: Code Questions

Consider the following code snippet related to text generation.

Snippet A:

```
1 sorted_p, sorted_i = torch.sort(probs, descending=True)
2 c = torch.cumsum(sorted_p, dim=0)
3 keep = c < p
4 filtered_p = sorted_p[keep]
5 sampled = torch.multinomial(filtered_p, 1)
6 next_token = sorted_i[sampled]
```

Question 28 Code snippet A is intended to implement top- p (nucleus) sampling. The following statements are labeled A–E. Which option correctly identifies the set of true statements?

- A. `keep = c < p` can exclude the boundary token that first crosses p .
- B. This code can produce an empty candidate set.
- C. Final indexing in line 6 is incorrect.
- D. Top- p should never sort probabilities.
- E. Cumulative mass should be computed from unsorted probabilities.

- ☐ C only
- ☐ A, C, and D
- ☐ None of the other options is correct.
- ☐ D and E
- ☐ B and C
- ☒ A, B, and C
- ☐ A, B, C, and E
- ☐ A and B

Solution: In top- p sampling, one should sort probabilities (**D & E incorrect**) and keep the smallest prefix whose cumulative mass reaches or exceeds p , including the boundary token that first crosses p (**A correct**). With `keep = c < p`, if the top token already exceeds p , no token is kept and the candidate set becomes empty (**B correct**). Finally, `sampled` indexes into the filtered list `filtered_p`, so the code must remap through the filtered index set rather than directly indexing `sorted_i` (**C correct**).



Text Generation: Computation

The following tree of tokens in Figure 1 is relevant for the next few questions. The boxes represent tokens. Each arrow $a \xrightarrow{q} b$ represents the choice of next token being b given the token prefix ending at a . The probability of b given prefix ending at a is q . Only the highest probability tokens for each prefix are shown.

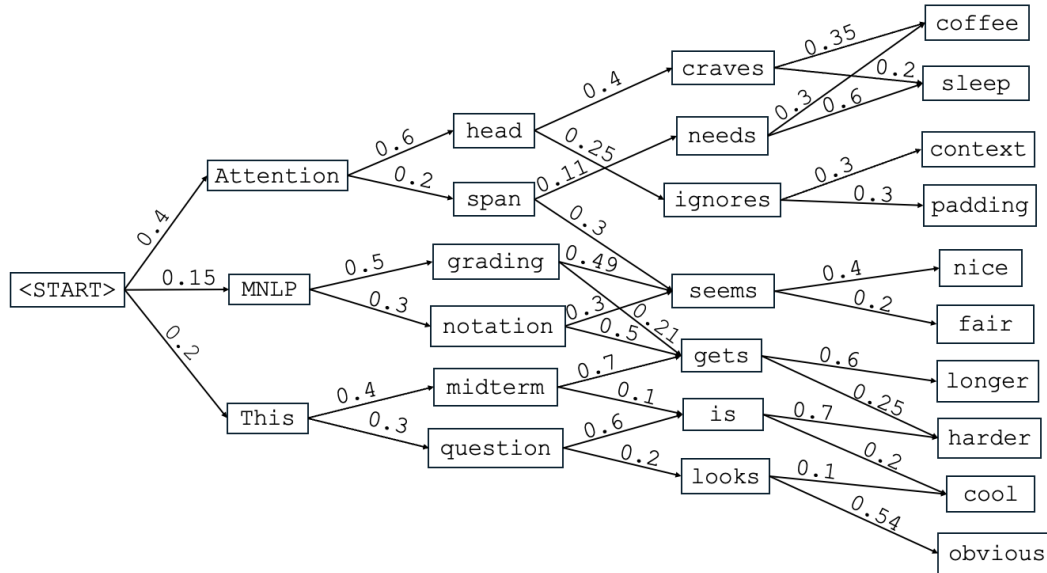


Figure 1. Tree of tokens and probabilities.



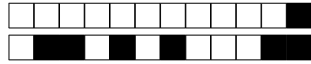
Question 29 Consider the following sequences:

- A. MNLP grading seems nice
- B. Attention head craves sleep
- C. MNLP notation gets longer
- D. Attention head ignores sleep
- E. This midterm gets harder
- F. This question is harder
- G. This midterm seems nice
- H. This question looks cool
- I. Attention span seems nice
- J. Attention head ignores context

Which of the above sequences will never be generated if top- p (nucleus) sampling is used for decoding with $p = 0.52$?

- ☐ A and C
- ☐ None of the sequences can be generated
- ☒ All except F, B, and J
- ☐ All except D, G, and I
- ☐ A, C, D, E, G, H, I, and J
- ☐ B, F, and J
- ☐ None of the other options is correct
- ☐ I, G, and D
- ☐ All the sequences can be generated
- ☐ I only

Solution: In top- p (nucleus) sampling, at each step, we sample from the smallest set of tokens whose cumulative probability $\geq p$. Any token outside this set cannot be selected, so any sequence that includes such a token at any step can never be generated. At the first step, only **Attention** and **This** are considered, given the cumulative probability of 0.6. Assuming **Attention** is sampled at the first step, only **head** is a candidate since it alone satisfies the cumulative probability criteria. Assuming **This** is sampled at the first step, both **midterm** and **question** are candidates with a cumulative probability of 0.7. Following this line of reasoning, we can prune the tree and conclude that only the sequences B, F, and J can be generated.



Question 30 What are the absolute values of differences in probabilities assigned to the following three sequences by top- k sampling with $k = 1$ and $k = 2$:

This midterm is cool
Attention span needs sleep
MNLP grading seems fair

- ☐ (0.0016, 0.00528, 0)
- ☐ (0.0016, 0.00528, 0.00735)
- ☒ ($\frac{1}{189}$, $\frac{11}{369}$, 0)
- ☐ (0 , $\frac{11}{369}$, 0)
- ☐ (0.0298, 0.0053, 0)
- ☐ ($\frac{1}{210}$, $\frac{11}{410}$, 0)
- ☐ ($\frac{4}{945}$, $\frac{44}{1845}$, $\frac{7}{240}$)

Solution: At each step in top- k sampling, we select top- k tokens and then normalize their probabilities. Doing so, we get that the probability assigned by top-2 sampling to **This midterm is cool** is $1/3 * 4/7 * 1/8 * 2/9 = 1/189$. **Attention span needs sleep** gets assigned the probability $2/3 * 1/4 * 11/41 * 2/3 = 11/369$. **MNLP grading seems fair** cannot be produced as it gets cut off in the very first step, so it is assigned a probability of 0. Note that top-1 sampling is simply greedy decoding and it selects **Attention head craves coffee**. All other sequences get a probability of 0 under it. Therefore, the absolute value of difference in probabilities is simply the top-2 sampling probability.